

DEVELOPMENT ENVIRONMENTS

Branches on GitHub

Raúl Palao



Secure isolation of new features

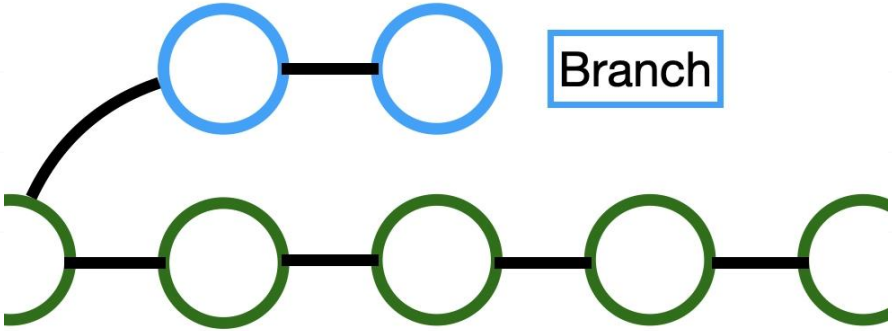
KEY CONCEPT

Branching is a fundamental concept in Git that allows us to isolate the development of new features or fixes.

Main Purpose

Ensure that the code under development does not directly impact the main project.

It allows multiple programmers to work simultaneously on different features without interfering with the stable code of other colleagues.



Basic structure of a standard project

In every professional project, we will work with a minimum of three types of branches:

Master Production

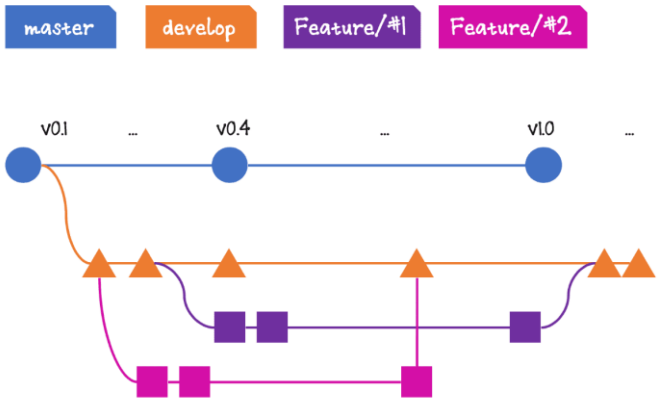
It is the main branch with the code in Production. The code is never touched directly in this branch; it is always integrated from Develop.

Develop Pre-production

Secondary branch for code in Pre-production. It is created from Master and receives the integrations of new functionalities.

Feature Temporary Work

Temporary branches. These are created by each programmer from within Develop to make changes or add new features. Upon completion, they are integrated back into Develop.

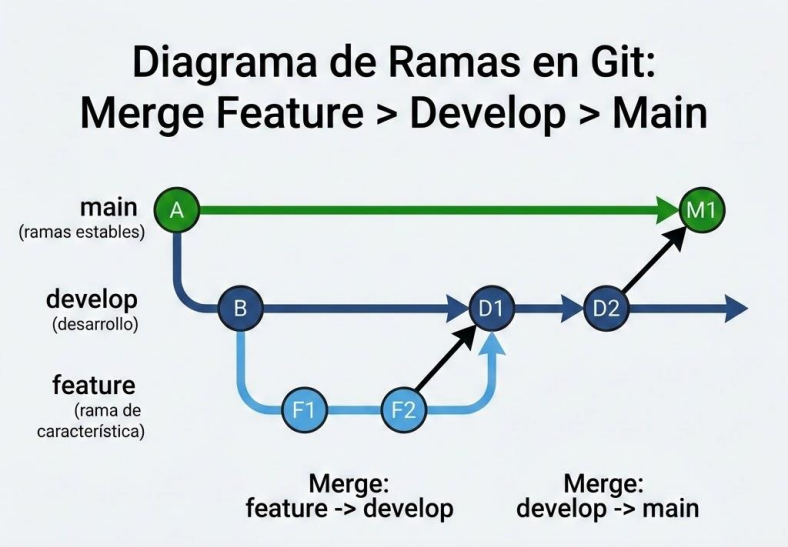


Merging of changes to the upper branch

Concept of Integration

The integration, executed using the command `git merge.` It consists of merging the confirmed changes from a lower-level branch to a higher-level branch in the project hierarchy.

This structured process allows developers' changes to be rolled out in stages: first from individual branches (Feature) to the integration environment (Develop), and finally, once validated, to the production environment (Master).



Repository status inspection

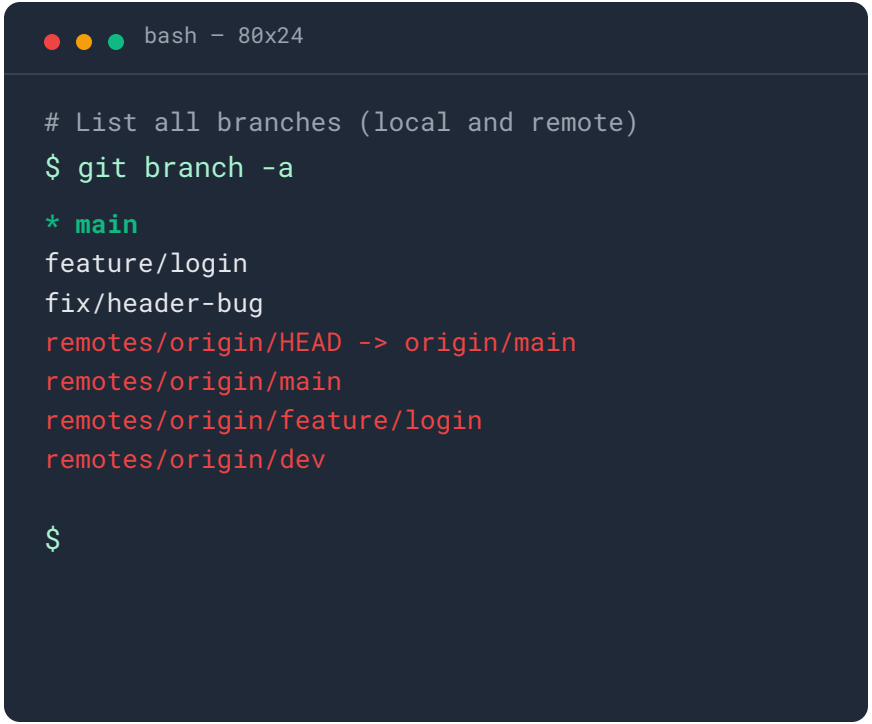
Project Structure

To understand the current structure of our project, Git provides branch visualization tools.

The fundamental command is `git branch -a`.

Executing this instruction in the terminal will list all existing branches associated with the project.

This includes both the branches we have downloaded in our local environment and the remote branches hosted on the server (for example, on GitHub).



Workspace management

Creating and moving between branches requires two essential commands:

Creation

```
$ git branch NAME
```

The command `git branch NAME` creates a new branch.

It is vital to remember that the new branch is always created as an exact copy of the code from the branch from which we executed the command.

Navigation

```
$ git checkout NAME
```

The command `git checkout NAME` allows us to move or "jump" to the specified branch.

Update the files in our working directory to reflect the exact state of that branch.

Start of the development cycle

To add a new feature, the programmer must follow a strict workflow:

1. Make sure you are on the correct base branch (**Develop**).
2. Create a new branch exclusively for the task, commonly called feature/task_name.
3. Move to the new branch to start programming.
4. It is a recommended practice to inform the team by pushing this new branch to the remote repository on GitHub using git push origin BRANCH_NAME.

bash — git workflow

```
# 1. Switch to the base branch
$ git checkout develop

Switched to branch 'develop'

# 2 and 3. Create and switch to the new branch
$ git checkout -b feature/1

Switched to a new branch 'feature/new-functionality'

# 4. Publish the branch on GitHub
$ git push origin feature/1

Total 0 (delta 0), reused 0 (delta 0)
To github.com:org/proyecto.git
* [new branch] feature/new-feature -> feature/new-
feature
Branch 'feature/new-functionality' set up to track
remote branch from 'origin'.
```

Confirmation and code upload

Once the files in our Feature branch have been modified, the save flow is as follows:that we already know:

Step 01 • Status

`git status`

It allows you to see which files have been modified in the working directory.

Step 02 • Staging

`git add <file>`

Add the modified files to the staging area.

Step 03 • Confirmation

`git commit -m "Message"`

Confirm the changes locally, packaging them with a descriptive message of the action taken.

Step 04 • Remote

`git push origin BRANCH_NAME`

Upload local commits to the remote repository in the cloud.

Code merging in pre-production

To incorporate the changes from our Feature branch into the integration environment (Develop), we must execute a precise sequence:

```
# 1. Move to the destination branch:
$ git checkout develop

# 2. Update the local branch with the changes from the cloud (Mandatory step!):
$ git pull origin develop

# 3. Incorporate our functionality:
$ git merge feature1

# 4. Upload the final merged result to the cloud:
$ git push origin develop
```

Deployment of the final version

Once the code is in Develop, it has been tested and validated. The final integration into Production (Master) is identical to the previous one, but changing the branches involved:

1. Switch to master branch: git checkout master
2. Synchronize with the remote repository: git pull origin master
3. Merge the validated code: git merge develop
4. Publish the new version to production: git push origin master

bash – master deployment

```
# 1. Switch to the main branch
$ git checkout master

# 2. Synchronize with the remote repository
$ git pull origin master

# 3. Merge the validated code
$ git merge develop

# 4. Release the new version to production
$ git push origin master
```

Repository maintenance

After successfully integrating a branch Feature in Master, and to validate its functionality, the original branch is no longer needed. To keep the repository clean and organized, it should be deleted.

01. Local Repository

Local Deletion

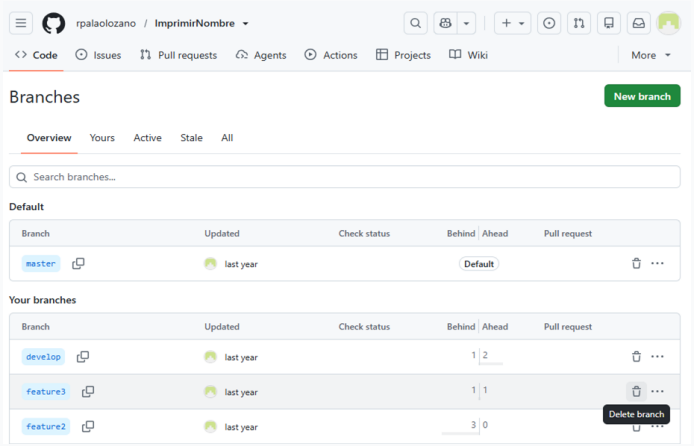
In the local terminal, we use the corresponding Git command to delete the branch from our machine once it has been merged.

```
# Delete branch locally
$ git branch -d branch_name
Deleted branch branch_name (was a1b2c3d).
```

02. Remote Repository

Remote Deletion

Subsequently, we must also remove the reference to the branch in the remote repository (the cloud) through the GitHub interface.



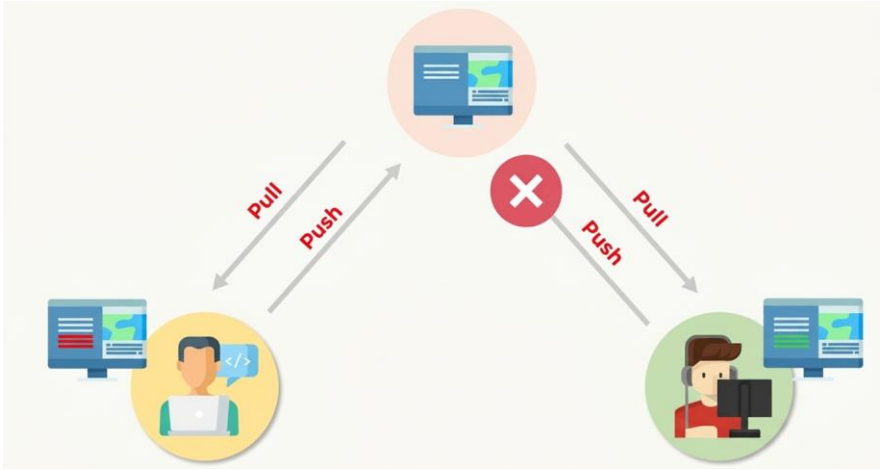
Origins of code collision

What is a conflict?

Merger conflicts arise when Git cannot automatically figure out how to combine different lines of code.

Main cause

This occurs when two or more users simultaneously manipulate the same file within the same branch.



Routine detection

It is detected when trying to do a `push` to the server and be rejected because there are changes in the cloud not present locally (usually due to forgetting to do a `pull` prior to the branch change).

Manual collision resolution

When a conflict occurs after a `pull`, Git injects markers directly into the affected file:

`<<<<<<< HEAD`
Indicates the start of our local code version.

`=======`
It acts as a separator between both versions.

`>>>>>>> id_commit`
Indicates the end of the version coming from the cloud.

Resolution procedure

To resolve this, the developer must edit the file, delete the unwanted lines (including Git bookmarks), and perform a new cycle: `add`, `commit` and `push`.

index.js – merge conflict

```
// File with unresolved conflict

function calculateTotal(price, tax) {

<<<<<<< HEAD
// Local changes (your version)
const total = price * (1 + tax);
return total.toFixed(2);

=======
// Remote changes (server)
return price + (price * tax);
>>>>>>> 4a8f9b2c

}
```

Reversing errors in the workflow

Git offers security mechanisms to undo actions using the command `reset`:

Reset in Staging (ADD)

If we add a file by mistake with `git add`, we can remove it from the preparation area using the reset command.

```
# Add file by mistake
$ git add archivo.txt
# Remove from Staging
$ git reset file.txt
```

Reset on Commit

To undo commits and return to a previous state of the project, we can either go back one commit or travel to a specific point in the history.

```
# Roll back one commit
$ git reset --hard HEAD~1
# Go to a specific commit
$git reset --hard <id_commit>
```